G1-412837 Master Dossier

Version: 1.1

Prepared on: 2026-04-13 (America/New_York)

Repository root: /home/chewy/projects/trading-compose-dev

Primary runtime: g1_412837_runtime_v1/runtime_g1_412837_loop.py

1) Purpose and Scope

This document is the single deep-reference dossier for G1-412837.

It is intended for strategy review, architecture review, operations handoff, and controlled change planning.

It covers:

- Base profile lineage
- Runtime architecture and code-level behavior
- Strategy rules and parameterization
- Intraday + end-of-day execution behavior
- Practical operating behavior
- Backtest/runtime-like result artifacts and interpretation
- Risk controls, known limitations, and recommended improvement workflow

It does not replace broker/legal/regulatory documentation.

2) Executive Summary

G1-412837 is a hybrid runtime variant that combines:

- A **fast-entry branch** (`fev1`) target proposal
- An **overlay branch** (OV, defensive SOXS regime logic)
- A **continuous blend weight** computed from SOXL rolling features using a sigmoid model

Then it executes with:

- Intraday profit-lock checks (5-minute cadence from the inherited base profile)
- Main rebalance at eval time (default `15:56` NY)
- Market or bracket rebalance orders (default market)
- Profit-lock order mode configurable (default market_order)

In observed runtime-like report artifacts for windows ending 2026-04-10, G1-412837 outperformed neighboring family variants on medium/long windows while often improving drawdown vs C_sp4.7 baseline.

---

3) Canonical Code and Artifacts

3.1 Runtime code

- `g1_412837_runtime_v1/runtime_g1_412837_loop.py`

3.2 Upstream/base dependencies (read-only lineage)

- `switch_runtime_v1/runtime_switch_loop.py`
- `csp47_overlay_research_v1/tools/sweep_csp47_overlays.py`
- `protective_stop_variant_v2/tools/export_last30_daybyday.py`
- `fast_entry_variant_v1/tools/fast_entry_override_grid.py`
- `soxl_growth/composer_port/symphony_soxl_growth_v245_rl.py`
- `soxl_growth/execution/*`, `soxl_growth/data/*`, `soxl_growth/db.py`

3.3 Primary result artifacts used in this dossier

- `protective_stop_variant_v2/reports/compare_G1_570248_vs_G1_412837_runtime_like_1m_2y_10k_end_2026-04-10.csv`
- `protective_stop_variant_v2/reports/compare_G1_412837_vs_C_sp4.7_rv75_tr1.10_th1.20_runtime_like_1m_2y_10k_end_2026-04-10.csv`
- `protective_stop_variant_v2/reports/compare_5_variants_summary_10k_2025-04-10_to_2026-04-10.csv`
- `protective_stop_variant_v2/reports/G1-412837_daybyday_10k_2025-04-10_to_2026-04-10_with_sell_buy_prices.csv`
- `protective_stop_variant_v2/reports/G1-412837_daybyday_with_sell_buy_prices_2026-03-10_to_2026-04-10_with_sell_buy_prices.csv`
- `hybrid_c_ov_research_v1/reports_gpu1/gpu_search_summary_local_seed9010.json`

---

4) Base Strategy Lineage

G1-412837 is not an isolated strategy. It is layered on top of the existing runtime family:

1. Base profile from `switch_runtime_v1/runtime_switch_loop.py`:
   - `aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m`
2. Profit-lock scaling and profile lock handling from CSP4.7 overlay tooling
3. Fast-entry (FEV1) and OV branch target generation paths
4. Final hybrid blend and runtime execution cycle in G1 runtime file

4.1 Base profile details (inherited)

From switch_runtime_v1/runtime_switch_loop.py:

- `profit_lock_mode = trailing`

- `profit_lock_threshold_pct = 10.0`
- `profit_lock_trail_pct = 2.0`
- `profit_lock_adaptive_threshold = true`
- adaptive symbol = `TQQQ`
- adaptive RV window = `14`
- adaptive baseline RV = `85`
- adaptive min/max threshold = `8` / `30`
- `intraday_profit_lock_check_minutes = 5`

These are then scaled in G1 flow with:

- `trail_scale = 1.10`
- `threshold_scale = 1.20`

4.2 Full Deep Dive: `aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m`

This subsection explains the profile in full detail.

Important: this profile controls risk/exit behavior and cadence, not the core symbol decision tree by itself.

4.2.1 Exact profile fields (as coded)

From switch_runtime_v1/runtime_switch_loop.py:

```
| Field | Value | Functional meaning |
|---|---|---|
| `name` | `aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m` | Profile identifier used in logs/events/CLI |
| `enable_profit_lock` | `true` | Enables intraday+eval-time profit-lock checks |
| `profit_lock_mode` | `trailing` | Uses trigger + trailing confirmation (not immediate fixed close) |
| `profit_lock_threshold_pct` | `10.0` | Base trigger threshold (%) above previous close |
| `profit_lock_trail_pct` | `2.0` | Trail distance (%) from intraday high once triggered |
| `profit_lock_adaptive_threshold` | `true` | Trigger threshold becomes volatility-adaptive |
| `profit_lock_adaptive_symbol` | `TQQQ` | Symbol whose realized vol drives adaptation |
| `profit_lock_adaptive_rv_window` | `14` | 14-day RV lookback window |
| `profit_lock_adaptive_rv_baseline_pct` | `85.0` | Baseline RV reference for scaling |
| `profit_lock_adaptive_min_threshold_pct` | `8.0` | Lower clamp for adaptive threshold |
| `profit_lock_adaptive_max_threshold_pct` | `30.0` | Upper clamp for adaptive threshold |
| `intraday_profit_lock_check_minutes` | `5` | Intraday check cadence every 5 minutes |
```

4.2.2 Naming breakdown (aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m)

Human decode:

- `aggr`: aggressive profile family
- `adapt`: adaptive threshold enabled
- `t10`: base threshold 10%
- `tr2`: trailing stop distance 2%
- `rv14`: realized-vol window = 14 days
- `b85`: RV baseline = 85
- `m8_M30`: min threshold 8%, max threshold 30%

- `intraday_pl_5m`: intraday profit-lock checks every 5 minutes

4.2.3 Adaptive threshold math (exact behavior)

Code path: _current_threshold_pct(profile, daily_closes)

Let:

- `base = 10`
- `rv = annualized realized volatility (%)` of `TQQQ` over prior 14 trading returns
- `baseline = 85`
- `ratio = rv / baseline`
- `raw = base * ratio`
- `threshold = clamp(raw, 8, 30)`

Result:

- If RV is low, threshold can compress toward 8%
- If RV is high, threshold can expand toward 30%
- If insufficient history (`len < window+1`), fallback is base 10%

Implication:

- High-vol regime demands larger move before profit-lock can arm.
- Low-vol regime allows earlier arming.

4.2.4 Profit-lock trigger/confirmation logic (exact)

Code path: _build_profit_lock_signals(...)

For each long position:

1. Compute `trigger_price = prev_close * (1 + threshold_pct/100)`
2. Require `day_high >= trigger_price` (arming condition)
3. Because mode is `trailing`:

- `trail_stop_price = day_high * (1 - trail_pct/100)` where trail_pct=2
- close signal if `last_price <= trail_stop_price`

So trailing mode needs both:

- prior intraday strength (crossing trigger)
- then retrace from high by trail amount

This avoids exiting purely on first threshold touch.

4.2.5 Intraday schedule and daily schedule interaction

Intraday:

- Every 5 minutes (slot-based idempotent keys) from market open until before eval-time.

- Each slot runs data pull + signal evaluation.
- Duplicate slot re-fire is prevented by saved slot key.

Daily eval-time:

- Main cycle runs once at eval-time (`15:55` in switch runtime baseline; `15:56` default in G1 runtime wrapper).
- Also does a profit-lock pass and rebalance pass.
- Once-per-day execution key prevents duplicate daily cycle.

4.2.6 Order-type interaction for this profile

Profile defines signal generation, while CLI defines execution style:

--profit-lock-order-type:

- `market_order`: immediate market sell on signal
- `close_position`: broker-side close call
- `stop_order`: submit stop based on trigger/trail reference (capped under market)
- `trailing_stop`: submit broker-native trailing stop order

--rebalance-order-type:

- `market` (default): direct market rebalance
- `bracket`: buy with attached TP/SL (if valid prices can be computed)

4.2.7 What this profile does NOT do

- It does not alone choose symbols via indicators; symbol selection comes from strategy evaluator / hybrid branch logic.
- It does not guarantee intraday exits every day; conditions must be met.
- It does not remove drawdowns; it shapes exit timing behavior.

4.2.8 Edge cases and safeguards

1. Missing/insufficient RV history:

- adaptive threshold falls back to base threshold.

2. Invalid price context:

- if prev close/day high/last price unavailable or non-positive, symbol is skipped for signal.

3. Stale intraday data:

- runtime can skip actions if data freshness exceeds threshold minutes.

4. Multiple intraday checks:

- deduplicated by slot key.

5. Order conflicts:

- optional cancellation of existing exit orders (`--cancel-existing-exit-orders`) before placing new ones.

4.2.9 Practical behavior profile owners should expect

- In trending high-momentum sessions:
- trigger may arm quickly; trailing rule may or may not confirm depending on pullback depth.
- In straight-line up days with little pullback:
- no trailing confirmation -> position can remain open.
- In volatile mean-reverting sessions:
- trigger+retrace can fire intraday and close earlier.
- At eval-time:
- strategy target can re-enter same ticker or switch ticker based on main target map.

4.2.10 Parameter sensitivity intuition

- Raising base threshold (`t10 -> higher`) generally reduces exit frequency.
- Increasing trail (`tr2 -> larger`) generally requires bigger pullback to confirm.
- Lowering RV baseline (85 -> lower) makes adaptive ratio larger for same RV, pushing threshold upward more often.
- Increasing min threshold (`m8`) removes low-vol early-exit behavior.
- Decreasing max threshold (`M30`) caps high-vol threshold growth, making exits easier to arm in extreme RV.

4.2.11 Relationship to G1 scaling

In G1 runtime wrapper, this profile is scaled:

- threshold multiplied by `threshold_scale=1.20`
- trail multiplied by `trail_scale=1.10`

So effective base behavior in G1 wrapper is stricter than raw profile defaults:

- threshold side is pushed upward
- trailing offset is widened

This is intentional in the C_sp4.7 family lock.

---

5) G1-412837 Strategy Design

5.1 High-level structure

G1 computes three elements each cycle:

- `C branch`: fast-entry based target map (`fev1`)
- `OV branch`: overlay target map with defensive branching (`SOXS` capable)
- `w_c`: a continuous confidence weight in `[floor, ceil]` from SOXL feature model

Final target:

- If C and OV pick the same symbol -> 100% that symbol
- If different -> allocation split:
- `C symbol = w_c`
- `OV symbol = 1 - w_c`

5.2 Feature model

Feature build function:

- `_rolling_features(close)` in `runtime_g1_412837_loop.py`

Lookahead policy:

- Uses data up to `t-1` when building day `t` features
- This is explicit in code (`p = i - 1`)

Features used:

- `mom20` (20d momentum, pct)
- `mom60` (60d momentum, pct)
- `rv20` (20d realized vol proxy, pct)
- `dd60` (60d rolling max drawdown, pct)

5.3 Weight equation

z = bias + a20*mom20 + a60*mom60 - b_rv*rv20 - b_dd*dd60

s = sigmoid(z / temp)

w_c = floor + (ceil - floor)*s, clamped to [0,1]

Locked candidate parameters (G1-412837)

```
| Parameter | Value |
|---|---:|
| bias | 6.787898344285149 |
| a20 | 0.0 |
| a60 | -0.09181296789807147 |
| b_rv | 0.09407960349884371 |
| b_dd | 0.13271417559907764 |
| temp | 0.03 |
| floor | 0.006844050593517129 |
| ceil | 1.0 |
```

Interpretation:

- Larger `rv20` and `dd60` reduce C branch weight
- Stronger `mom60` also reduces C weight due to negative `a60` (counter-trend tendency in this fitted candidate)
- Temperature is very low (`0.03`), making sigmoid near binary in many states

5.4 Branch sources

C branch

- Produced via `_build_targets_for_engine(engine="fev1", ...)`
- Input includes baseline-target context and control-plane hysteresis knobs

OV branch

- Fixed candidate:
- `shock_drop_pct=6`
- `shock_hold_days=1`
- `dd_trigger_pct=0`
- `dd_window_days=20`
- `reentry_pos_days=1`
- `defensive_symbol="SOXS"`
- Built via `_overlay_targets(...)`

Threshold behavior

- Daily adaptive threshold pct derived from base runtime helper `_current_threshold_pct(...)`
- Uses adaptive RV scaling bounds from profile

---

6) Runtime Architecture (End-to-End)

6.1 Components

```
| Layer | Component | Responsibility |
|---|---|---|
| Config | `soxl_growth.config` | API keys, feed, timezone config |
| Data | `AlpacaBarLoader` | Fetches 1Day adjusted bars and 1Min raw bars |
| Broker | `AlpacaBroker` | Clock/calendar/account/positions/orders |
| Strategy Runtime | `runtime_g1_412837_loop.py` | Signal + target + execution orchestration |
| Storage | `StateStore` | Idempotency keys, state snapshots, event logs |
| Orders | `build_rebalance_order_intents` | Converts target weights to side/qty intents |
```

6.2 Control loop chronology

1. Load optional env file
2. Initialize profile + scaled profile + params
3. Connect broker/loader/store
4. If market closed -> wait/exit
5. If pre-eval intraday slot -> run intraday profit-lock pass

6. If before eval time -> wait
7. If already executed today -> wait/exit
8. Build G1 target for today
9. Pull intraday stats + stale-data check
10. Evaluate/submit profit-lock exits
11. Build rebalance intents
12. Submit rebalance orders
13. Persist state keys + append cycle event

6.3 Idempotency and state keys

Key state entries:

- `g1_switch_executed_day`
- `g1_switch_intraday_profit_lock_last_slot`
- `g1_switch_last_profile`
- `g1_switch_last_variant`
- `g1_switch_last_final_target`

These prevent duplicate same-day cycles and duplicate intraday slot closes.

6.4 Event types written

- `g1_switch_profit_lock_intraday_close`
- `g1_switch_profit_lock_close`
- `g1_switch_rebalance_order`
- `g1_switch_cycle_complete`

Each includes timestamps and payload metadata for audit/replay.

---

7) Execution Semantics and Practical Behavior

7.1 Profit-lock evaluation

For each long position:

- `trigger_price = prev_close * (1 + threshold_pct)`
- Trigger requires `day_high >= trigger_price`
- For trailing mode, close signal only if `last_price <= day_high*(1-trail_pct)`

With market_order setting:

- Signal results in market sell order when signal exists

With stop_order or trailing_stop:

- Exit can persist as a resting order model
- Rebalance intents avoid symbols closed by certain exit modes in same pass

7.2 Intraday vs eval-time behavior

Intraday checks run every intraday_profit_lock_check_minutes (inherited 5m) before eval-time.

Main daily cycle runs at --eval-time (default 15:56 for G1 runtime).

7.3 Rebalance behavior

Target weights are converted to order intents using:

- Current equity
- Current qty
- Last intraday price
- Min trade delta threshold (`--rebalance-threshold`, default `0.05`)

Order styles:

- Market (default)
- Bracket for buys (optional) using configured TP/SL percentages

---

8) CLI and Operational Parameters

Core runtime command template:

```
/home/chewy/projects/trading-compose-dev/composer_original/.venv/bin/python \
  /home/chewy/projects/trading-compose-dev/g1_412837_runtime_v1/runtime_g1_412837_loop.py \
  --env-file /home/chewy/projects/trading-compose-dev/.env.dev \
  --env-override \
  --mode paper \
  --strategy-profile aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m \
  --data-feed sip \
  --eval-time 15:56 \
  --profit-lock-order-type market_order \
  --rebalance-order-type market \
  --execute-orders
```

Live is identical except --mode live.

Notable defaults in parser

```
| Parameter | Default |
|---|---|
| strategy profile | `aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m` |
| variant-id | `G1-412837` |
```

```
| eval-time | `15:56` |
| data-feed | `sip` |
| rebalance-threshold | `0.05` |
| trail-scale | `1.10` |
| threshold-scale | `1.20` |
| profit-lock-order-type | `market_order` |
| rebalance-order-type | `market` |
| bracket TP/SL | `12% / 6%` |
| warmup-days | `260` |
| daily-lookback-days | `800` |
```

8.5 Deep Code Walkthrough: `runtime_g1_412837_loop.py`

This section is a direct explanation of the runtime file internals, function-by-function.

8.5.1 Import and bootstrap block (lines 1-33)

Purpose:

- Makes the repository root importable by inserting `ROOT` into `sys.path`
- Imports hybrid dependencies:
- intraday verifier helpers (`composer_original.tools.intraday_profit_lock_verification`)
- CSP/OV overlay helpers (`csp47_overlay_research_v1`)
- FEV branch target helper (`protective_stop_variant_v2.tools.export_last30_daybyday`)
- base runtime helpers (`switch_runtime_v1.runtime_switch_loop`)
- Imports execution stack (`AlpacaBroker`, `AlpacaBarLoader`, `StateStore`, order intent builder)

Why this matters:

- G1 runtime is intentionally **thin orchestration + custom blend math**, while reusing proven data/execution helpers from existing modules.

8.5.2 Parameter dataclass: `G1Params` (lines 36-47)

G1Params is the locked candidate parameter object.

Design intent:

- Keep tuned weights centralized and explicit
- Allow CLI overrides only when intentional (`--g1-*` flags)
- Keep defaults aligned with selected research candidate

8.5.3 Feature engineering function: `_rolling_features` (lines 49-84)

Inputs:

- `close: list[float]` (SOXL daily closes)

Outputs:

- `mom20, mom60, rv20, dd60` arrays aligned to each day index

Implementation detail:

- For index `i`, the function uses `p = i - 1`, so all features use prior data and avoid lookahead.
- `rv20` is standard deviation of 20 daily returns in percent units.
- `dd60` is rolling drawdown over 60-day close window.

8.5.4 Weight model: `_weight_for_day` (lines 86-100)

Pipeline:

1. Linear score `z` using `G1Params` and feature values
2. Temperature-scaled sigmoid
3. Clamp to `[floor, ceil]`
4. Clamp final to `[0,1]`

Operational meaning:

- This is the “confidence-like” blending control between C and OV branches.

8.5.5 Target normalization helpers (lines 102-120)

- `_pick_top_symbol(target)`:
- Returns max-weight ticker, or `CASH` if empty map
- `_compose_target(c_sym, o_sym, weight_c)`:
- Handles all symbol-combination edge cases
- Returns a valid target-weight dictionary

Edge cases handled explicitly:

- CASH/CASH => empty target
- one side CASH => 100% non-cash side
- same ticker => 100%
- different tickers => split using `weight_c`

8.5.6 Core target builder: `_build_g1_target_for_today` (lines 122-210)

This is the most important strategy function in the file.

Step-by-step:

1. Align daily close history across all symbols (`_align_daily_close_history`)
2. Build baseline by-day target map (`_build_baseline_target_by_day`)
3. Build C branch targets via `_build_targets_for_engine(engine="fev1", ...)`
4. Build OV branch targets via fixed `OverlayCandidate(6,1,0,20,1,"SOXS")`
5. Select latest aligned day `<= today`
6. Build SOXL features and compute `weight_c`

7. Select top C ticker and top OV ticker

8. Compose final target map with `_compose_target`

9. Compute adaptive threshold using base runtime helper `_current_threshold_pct`

10. Return:

- final target weights
- threshold pct
- diagnostics payload (`aligned_day`, `weight_c`, branch picks, final target)

8.5.7 Runtime loop: `_run_loop` (lines 213-550)

This function is the production control loop.

Initialization phase

- Loads env vars if `--env-file` provided
- Reads selected base profile from `base_rt.PROFILES`
- Applies profile scaling with `_build_scaled_profile(trail_scale, threshold_scale)`
- Converts locked profile back to `StrategyProfile` format for helper compatibility
- Constructs `params` from `--g1-*` args
- Initializes:
- `AlpacaConfig` from env
- `StateStore`
- `AlpacaBroker`
- `AlpacaBarLoader`
- symbol universe from `StrategyConfig`

Per-loop cycle phase

- Clock/market-open check
- Session window derivation (`_market_session_window`)
- Intraday slot handling:
- computes slot key by minute bucket
- prevents duplicate slot execution using `g1_switch_intraday_profit_lock_last_slot`
- fetches daily + intraday stats
- stale-data guard
- builds and optionally submits intraday profit-lock orders
- Eval gate:
- waits until `--eval-time`
- enforces once-per-day with `g1_switch_executed_day`
- Main G1 decision cycle:
- fetch daily OHLC

- compute `target_weights` + `threshold_pct` + diagnostics
- fetch intraday day stats and stale-data check
- build/submit profit-lock signals
- refresh positions when needed
- get account equity
- build rebalance intents
- optionally block certain rebalance intents when stop/trailing orders are active
- optional cap by `--max-intents-per-cycle`
- submit market/bracket orders
- Persistence:
- writes last day/profile/variant/target keys
- appends `g1_switch_cycle_complete` event
- prints cycle JSON for external logging capture

8.5.8 Order submission behavior inside G1 loop (lines 476-523)

Per intent:

- quantity sanity checks (`sell` capped by current position qty)
- if rebalance mode is bracket + buy:
- computes TP/SL from current last price
- submits bracket only if both prices valid
- falls back to market order otherwise
- appends `g1_switch_rebalance_order` event with:
- symbol, side, qty, target weight
- order type, TP/SL prices
- variant/profile metadata

8.5.9 Parser and CLI contract: `_build_parser` (lines 552-620)

Argument families:

1. Runtime mode/environment:

- `--mode {paper,live}`
- `--env-file`, `--env-override`
- `--execute-orders`, `--run-once`

2. Scheduling/data:

- `--eval-time`
- `--data-feed`

- `--daily-lookback-days`, `--warmup-days`
- `--stale-data-threshold-minutes`
- `--loop-sleep-seconds`

3. Risk/controlplane:

- `--rebalance-threshold`
- `--controlplane-threshold-cap`
- hysteresis enter/exit values and days

4. Profile scaling:

- `--trail-scale`
- `--threshold-scale`

5. G1 model parameters:

- `--g1-bias`, `--g1-a20`, `--g1-a60`, `--g1-b-rv`, `--g1-b-dd`
- `--g1-temp`, `--g1-floor`, `--g1-ceil`

6. Execution style:

- `--profit-lock-order-type`
- `--cancel-existing-exit-orders`
- `--stop-price-offset-bps`
- `--rebalance-order-type`
- bracket TP/SL values

8.5.10 Error handling and process exit

main() wraps _run_loop and returns:

- `130` on keyboard interrupt
- `1` on unhandled exception (with stack trace via logger)
- `0` on normal completion (`--run-once` or graceful closed-market exit conditions)

8.5.11 Practical “mental model” of this file

You can think of runtime_g1_412837_loop.py as:

- A deterministic **daily orchestration engine**
- With periodic intraday exit checks
- That computes one hybrid target map per day
- Then executes real broker intents with strict idempotency and event logging

In short:

- **Target generation = hybrid model math**
- **Execution = broker/order pipeline**

- **Safety = stale checks + once-per-day keys + event audit trail**

---

9) Research Provenance and Candidate Selection

Source:

- `hybrid_c_ov_research_v1/reports_gpu1/gpu_search_summary_local_seed9010.json`

Key details:

- GPU device: `NVIDIA GeForce RTX 5060 Ti`
- Proxy candidates evaluated: `1,200,000`
- Full candidates validated: `100`
- `G1-412837` recorded as nearest gate candidate with:
- 1m..2y return/dd vector matching deployed candidate parameters

This establishes that G1 was not arbitrary; it came from a constrained search process.

---

10) Results (Artifact-Based)

All tables below are directly from listed report artifacts.

10.1 Runtime-like comparison: G1 vs G1-570248 vs C_sp4.7

Windows ending 2026-04-10, start equity 10,000 per window.

Variant	Window	Final Equity	Return %	MaxDD %
C_sp4.7_rv75_tr1.10_th1.20	1m	20,742.69	107.43	16.73
G1-412837	1m	20,742.69	107.43	16.73
G1-570248	1m	20,742.69	107.43	16.73
C_sp4.7_rv75_tr1.10_th1.20	2m	27,349.90	173.50	24.45
G1-412837	2m	27,285.14	172.85	16.73
G1-570248	2m	27,285.70	172.86	16.73
C_sp4.7_rv75_tr1.10_th1.20	3m	31,511.49	215.11	33.31
G1-412837	3m	31,436.88	214.37	24.69
G1-570248	3m	31,437.52	214.38	24.69
C_sp4.7_rv75_tr1.10_th1.20	4m	36,226.02	262.26	56.23
G1-412837	4m	36,140.22	261.40	28.08
G1-570248	4m	36,140.96	261.41	28.08
C_sp4.7_rv75_tr1.10_th1.20	5m	52,668.81	426.69	66.20
G1-412837	5m	70,214.09	602.14	28.08
G1-570248	5m	62,796.30	527.96	29.87
C_sp4.7_rv75_tr1.10_th1.20	6m	42,749.84	327.50	66.20
G1-412837	6m	68,276.72	582.77	31.08
G1-570248	6m	61,009.27	510.09	31.24
C_sp4.7_rv75_tr1.10_th1.20	1y	93,383.01	833.83	78.50
G1-412837	1y	157,718.59	1477.19	61.70
G1-570248	1y	140,930.79	1309.31	61.78
C_sp4.7_rv75_tr1.10_th1.20	2y	39,523.89	295.24	81.52

```
| G1-412837 | 2y | 108,471.43 | 984.71 | 71.09 |
| G1-570248 | 2y | 96,705.86 | 867.06 | 71.16 |
```

Interpretation

- G1 dominates CSP4.7 in medium-long windows with much lower DD.
- G1 outperforms G1-570248 on 5m, 6m, 1y, 2y in this artifact set.
- Short windows 1m-4m are similar across candidates, with DD benefit for G1 vs C_sp4.7.

10.2 One-year summary across five variants (2025-04-10 to 2026-04-10)

```
| Variant | Start | Final | PnL | Return % | MaxDD % | Days |
|---|---:|---:|---:|---:|---:|---:|
| G1-412837 | 10,000.00 | 144,385.47 | 134,385.47 | 1343.85 | 28.08 | 251 |
| G1-570248 | 10,000.00 | 129,032.44 | 119,032.44 | 1190.32 | 28.08 | 251 |
| C_sp4.7_rv75_tr1.10_th1.20 | 10,000.00 | 89,484.12 | 79,484.12 | 794.84 | 32.78 | 251 |
| OV_sh6_h1_dd0w20_re1_SOXS | 10,000.00 | 72,874.95 | 62,874.95 | 628.75 | 26.55 | 251 |
| OV_sh6_h1_dd0w40_re1_SOXS | 10,000.00 | 72,874.95 | 62,874.95 | 628.75 | 26.55 | 251 |
```

10.3 G1 day-by-day operational profile (2025-04-10 to 2026-04-10)

From G1-412837_daybyday_10k_2025-04-10_to_2026-04-10_with_sell_buy_prices.csv:

- Trading days: `251`
- Start equity: `10,000.00`
- Final equity: `144,385.47`
- Max drawdown: `28.08%`
- Best day PnL: `+17,401.28` on `2026-04-08`
- Worst day PnL: `-10,902.95` on `2026-03-30`
- Days with intraday switch event: `37`
- Days with explicit buy event logged: `67`

End-of-day ticker concentration:

- SOXL: 194 days
- SOXS: 22 days
- SPXL: 16 days
- Others (SPXS/TMV/SQQQ/CASH/TQQQ/TMF): remainder

10.4 Month-by-month equity progression (same 1-year artifact)

```
| Month | Start Eq | End Eq | PnL | Return % | MaxDD % (month) |
|---|---:|---:|---:|---:|---:|
| 2025-04 | 10,000.00 | 9,617.37 | -382.63 | -3.83 | 14.07 |
| 2025-05 | 9,617.37 | 10,580.02 | 962.65 | 10.01 | 14.43 |
| 2025-06 | 10,580.02 | 13,506.69 | 2,926.67 | 27.66 | 9.73 |
| 2025-07 | 13,506.69 | 12,031.48 | -1,475.21 | -10.92 | 18.74 |
| 2025-08 | 12,031.48 | 12,797.16 | 765.68 | 6.36 | 20.52 |
| 2025-09 | 12,797.16 | 17,205.57 | 4,408.41 | 34.45 | 17.13 |
```

```
| 2025-10 | 17,205.57 | 19,929.58 | 2,724.01 | 15.83 | 20.26 |
| 2025-11 | 19,929.58 | 34,206.98 | 14,277.40 | 71.64 | 17.26 |
| 2025-12 | 34,206.98 | 35,965.34 | 1,758.36 | 5.14 | 28.08 |
| 2026-01 | 35,965.34 | 53,178.70 | 17,213.36 | 47.86 | 12.08 |
| 2026-02 | 53,178.70 | 51,498.14 | -1,680.56 | -3.16 | 24.69 |
| 2026-03 | 51,498.14 | 82,325.76 | 30,827.62 | 59.86 | 16.73 |
| 2026-04 | 82,325.76 | 144,385.47 | 62,059.71 | 75.38 | 1.48 |
```

---

11) Practical Behavior Observed

11.1 Why intraday switch can appear while ticker ends unchanged

The log can show an intraday sell event and later a buy such that start ticker and end ticker match.

This means:

- Intraday profit-lock closed/reduced risk at signal time
- End-of-day rebalance target still pointed to same ticker, so it was repurchased

Operationally this is expected in strong trend + high intraday volatility conditions.

11.2 Why drawdowns still occur despite intraday controls

- Profit-lock requires both threshold trigger and trailing condition
- If no trigger is hit, position can ride daily move
- If move gaps and then trends down before favorable trigger, drawdown can deepen
- Re-entry at eval-time can re-risk quickly if target model remains aggressive

---

12) Detailed Parameter and Rule Matrix

```
| Category | G1-412837 Setting | Source |
|---|---|---|
| Base strategy profile | `aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m` | runtime args default |
| Variant id | `G1-412837` | runtime args default |
| Eval time | `15:56 NY` | parser default |
| Data feed | `sip` | parser default |
| Intraday check cadence | 5 minutes (inherited profile) | base profile |
| Profit-lock mode | trailing | inherited profile |
| Profit-lock threshold base | 10% (adaptive-scaled) | base profile + scaling |
| Profit-lock trail base | 2% (scaled) | base profile + scaling |
| Threshold scaling | `1.20` | parser default |
| Trail scaling | `1.10` | parser default |
| Rebalance min delta | `0.05` | parser default |
| C branch engine | `fev1` | `_build_targets_for_engine` call |
| OV branch | `sh6 h1 dd0w20 re1 SOXS` | OverlayCandidate in code |
| Final blend | sigmoid-weight mix C vs OV | `_compose_target` |
| Order type (profit lock) | `market_order` (default) | parser |
| Order type (rebalance) | `market` (default) | parser |
| Bracket TP/SL | 12% / 6% (if bracket mode selected) | parser |
| Stale data guard | 3 min default | parser |
```

```
| Warmup days | 260 | parser |
| Daily lookback days | 800 | parser |
```

---

13) Architecture Diagram (Textual)

```
[Alpaca Clock/Calendar]
        |
        v
[Session gate + eval-time gate] -----> [StateStore idempotency checks]
        |
        +--> (intraday slots every 5m before eval)
        |       |
        |       v
        |   [Fetch 1Day + 1Min bars] -> [adaptive threshold] -> [profit-lock signals] -> [optional exits]
        |
        v
[Main eval-time cycle]
        |
        +--> [Fetch 1Day bars] -> [C targets via fev1]
        |                         [OV targets via overlay candidate]
        |                         [SOXL features -> sigmoid weight]
        |                         [compose final target map]
        |
        +--> [Fetch 1Min bars + stale check]
        |
        +--> [profit-lock signal pass]
        |
        +--> [rebalance intents from target + current positions]
        |
        +--> [submit market/bracket orders]
        |
        +--> [persist state keys + append cycle events]
```

---

14) Data/Result Integrity Notes

There are multiple similarly named report files. Not all are authoritative:

- `G1-412837_runtime_like_1m_2m_3m_4m_5m_6m_1y_10k_end_2026-04-10.csv` exists in two forms; one artifact shows all zeros.
- This dossier used the **non-zero comparative files** and day-by-day files as canonical for quantitative claims.

Recommended governance:

1. Always tag run id + command + git SHA in output filenames.
2. Keep one “canonical” folder per experiment run.
3. Validate non-zero and monotonic date checks before publishing.

---

15) Risk, Limitations, and Operational Caveats

1. Runtime-like backtests are closer to live than pure daily synthetic, but not identical to real fills.

2. Real broker execution can diverge due to:

- spread/liquidity microstructure
- order queue/latency
- partial fills
- after-hours behavior differences

3. Threshold scaling and low sigmoid temperature create highly responsive allocations; this can amplify both gains and losses.

4. Risk is concentrated in leveraged ETFs; drawdowns can still be material.

---

16) Paper/Live Deployment Runbook

16.1 Preconditions

- Valid Alpaca credentials in `.env.dev`
- Correct data feed entitlement for `sip`
- State DB path chosen and backed up
- Dry run with `--run-once` before `--execute-orders`

16.2 Recommended paper command

```
/home/chewy/projects/trading-compose-dev/composer_original/.venv/bin/python \
  /home/chewy/projects/trading-compose-dev/g1_412837_runtime_v1/runtime_g1_412837_loop.py \
  --env-file /home/chewy/projects/trading-compose-dev/.env.dev \
  --env-override \
  --mode paper \
  --strategy-profile aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m \
  --data-feed sip \
  --eval-time 15:56 \
  --profit-lock-order-type market_order \
  --rebalance-order-type market \
  --execute-orders
```

16.3 Live command

Same as paper with --mode live.

16.4 Safety knobs

- `--max-intents-per-cycle`
- `--rebalance-threshold`
- `--stale-data-threshold-minutes`
- `--cancel-existing-exit-orders`

---

17) Controlled Improvement Plan (No Immediate Code Changes)

If reviewing for next iteration, prioritize:

1. **Stability pass**
   - run consistent A/B across `market_order`, `stop_order`, `trailing_stop`
   - compare realized slippage/fill quality
2. **Regime robustness**
   - stress test against high-volatility subwindows
   - inspect days with worst 5 PnL outcomes for trigger timing mismatch
3. **Model calibration**
   - explore slightly higher temperature or clipped weight transitions to reduce abrupt regime flips
   - preserve the best 1y-2y characteristics while reducing worst-month drawdown
4. **Operational analytics**
   - standardize event schema export for easier day-level root-cause analysis

---

18) Review Checklist for Stakeholders

- [ ] Do we accept the C/OV blended architecture as intended?
- [ ] Are G1 parameter locks and scaling assumptions approved?
- [ ] Is `15:56` eval time operationally correct for desk workflow?
- [ ] Are intraday check cadence and stale-data guard adequate?
- [ ] Is the order-type policy aligned with broker behavior?
- [ ] Are reported performance files traceable to command + commit?
- [ ] Are risk bounds (DD tolerance) acceptable for deployment?

---

19) Appendix A: Key Code Anchors

From g1_412837_runtime_v1/runtime_g1_412837_loop.py:

- `G1Params`: lines 36-47
- `_rolling_features`: lines 49-84
- `_weight_for_day`: lines 86-100
- `_compose_target`: lines 108-120

- `_build_g1_target_for_today`: lines 122-210
- Runtime loop `_run_loop`: lines 213-550
- Parser defaults `_build_parser`: lines 552-620

From switch_runtime_v1/runtime_switch_loop.py:

- `StrategyProfile`: line 33
- Base profile entry: lines 80-95
- Adaptive threshold `_current_threshold_pct`: lines 150-172
- Daily/intraday fetch helpers: lines 174-230
- Profit-lock signal build: lines 261-305
- Profit-lock order submission: lines 307-367

---

20) Appendix B: Exact Artifacts Referenced

- `/home/chewy/projects/trading-compose-dev/g1_412837_runtime_v1/runtime_g1_412837_loop.py`
- `/home/chewy/projects/trading-compose-dev/switch_runtime_v1/runtime_switch_loop.py`
- `/home/chewy/projects/trading-compose-dev/hybrid_c_ov_research_v1/reports_gpu1/gpu_search_summary_local_seed9010.json`
- `/home/chewy/projects/trading-compose-dev/protective_stop_variant_v2/reports/compare_G1_570248_vs_G1_412837_runtime_like_1m_2y_10k_end_2026-04-10.csv`
- `/home/chewy/projects/trading-compose-dev/protective_stop_variant_v2/reports/compare_G1_412837_vs_C_sp4.7_rv75_tr1.10_th1.20_runtime_like_1m_2y_10k_end_2026-04-10.csv`
- `/home/chewy/projects/trading-compose-dev/protective_stop_variant_v2/reports/compare_5_variants_summary_10k_2025-04-10_to_2026-04-10.csv`
- `/home/chewy/projects/trading-compose-dev/protective_stop_variant_v2/reports/G1-412837_daybyday_10k_2025-04-10_to_2026-04-10_with_sell_buy_prices.csv`
- `/home/chewy/projects/trading-compose-dev/protective_stop_variant_v2/reports/G1-412837_daybyday_with_sell_buy_prices_2026-03-10_to_2026-04-10_with_sell_buy_prices.csv`

---

21) Run Manifest and Reproducibility

Repo commit at dossier build time:

- `df6a91ae21e3a767331a98ec4179e7d7cbb29d5c`

21.1 Artifact integrity manifest

```
| Artifact | Size (bytes) | mtime (local) | SHA256 |
|---|---:|---|---|
| `protective_stop_variant_v2/reports/compare_G1_570248_vs_G1_412837_runtime_like_1m_2y_10k_end_2026-04-10.csv`
| `protective_stop_variant_v2/reports/compare_G1_412837_vs_C_sp4.7_rv75_tr1.10_th1.20_runtime_like_1m_2y_10k_end
| `protective_stop_variant_v2/reports/compare_5_variants_summary_10k_2025-04-10_to_2026-04-10.csv` | 366 | 2026-
```

```
| `protective_stop_variant_v2/reports/G1-412837_daybyday_10k_2025-04-10_to_2026-04-10_with_sell_buy_prices.csv`
| `protective_stop_variant_v2/reports/G1-412837_daybyday_with_sell_buy_prices_2026-03-10_to_2026-04-10_with_sell
| `hybrid_c_ov_research_v1/reports_gpu1/gpu_search_summary_local_seed9010.json` | 1660 | 2026-04-12T14:55:03.572
| `g1_412837_runtime_v1/runtime_g1_412837_loop.py` | 27019 | 2026-04-12T15:50:22.104719 | `0afb47a8171e46901e6e1
| `switch_runtime_v1/runtime_switch_loop.py` | 36767 | 2026-03-23T01:22:46.134424 | `6cef9d152d720a63fc2a6a9715b
```

21.2 Command manifest status

For several historical CSV artifacts, the original one-line command used at generation time is not embedded in those files.

Recommendation (mandatory for future runs): for every run, save:

- command
- start/end time
- git SHA
- input dataset hash
- output file hashes

Suggested run-manifest JSON schema:

```
{
  "run_id": "G1-YYYYMMDD-HHMMSS",
  "git_sha": "<commit>",
  "command": "<exact command>",
  "start_ts": "<iso8601>",
  "end_ts": "<iso8601>",
  "inputs": [{"path":"...", "sha256":"..."}],
  "outputs": [{"path":"...", "sha256":"..."}],
  "notes": "paper-like runtime backtest"
}
```

---

22) Data Provenance and Reconciliation

22.1 Canonical vs non-canonical artifacts

Canonical for this dossier:

- comparative runtime-like CSVs with non-zero values
- day-by-day with sell/buy price columns
- GPU search summary JSON

Non-canonical warning:

- a same-family file exists that contains all-zero window outputs:
- `protective_stop_variant_v2/reports/G1-412837_runtime_like_1m_2m_3m_4m_5m_6m_1y_10k_end_2026-04-10.csv`

Decision rule used here:

- When duplicated-named artifacts conflict, use:

1. file with consistent non-zero window behavior
2. file aligned with comparative reports and day-by-day totals
3. latest verified hash recorded in Section 21

22.2 Reconciliation checks run

- Cross-checked 1m..2y windows across comparative CSVs.
- Cross-checked 1y summary final equity vs day-by-day terminal equity.
- Verified G1 params in runtime code match GPU summary candidate.

---

23) State and Event Schema Appendix (Full)

State storage backend:

- `soxl_growth/db.py` (`state_kv` and `events` SQLite tables)

23.1 `state_kv` table

```
| Column | Type | Description |
|---|---|---|
| `key` | TEXT PK | state key |
| `value_json` | TEXT | JSON payload |
| `updated_at` | TEXT | UTC ISO update timestamp |
```

23.2 `events` table

```
| Column | Type | Description |
|---|---|---|
| `id` | INTEGER PK AUTOINCREMENT | event row id |
| `ts` | TEXT | event write timestamp (UTC ISO) |
| `event_type` | TEXT | event class |
| `payload_json` | TEXT | JSON payload |
```

23.3 G1-specific state keys

```
| Key | Meaning |
|---|---|
| `g1_switch_intraday_profit_lock_last_slot` | dedupe key for intraday slot evaluation |
| `g1_switch_executed_day` | dedupe key for once-per-day main cycle |
| `g1_switch_last_profile` | last active runtime profile name |
| `g1_switch_last_variant` | last variant id (`G1-412837`) |
| `g1_switch_last_final_target` | latest computed target-weight map |
```

23.4 G1 event types and payload contracts

g1_switch_profit_lock_intraday_close

Base payload fields:

- `ts`, `symbol`, `qty`, `profile`, `threshold_pct`, `profit_lock_order_type`
- `trigger_price`, `trail_stop_price`, `last_price`, `cancelled_open_orders`

Additional:

- `intraday_slot`, `variant_id`

g1_switch_profit_lock_close

Base payload fields:

- same as above

Additional:

- `variant_id`

g1_switch_rebalance_order

Fields:

- `ts`, `symbol`, `side`, `qty`, `target_weight`
- `profile`, `variant`, `order_type`
- `take_profit_price`, `stop_loss_price`

g1_switch_cycle_complete

Fields:

- `ts`, `day`, `profile`, `variant`
- `threshold_pct`
- `profit_lock_closed_symbols`
- `profit_lock_order_type`, `rebalance_order_type`
- `intent_count`, `orders_submitted`, `execute_orders`
- `g1_diag` (includes aligned day, weight_c, c_symbol, ov_symbol, final_target)

23.5 Example cycle-complete payload

```
{
  "ts": "2026-04-10T15:56:30-04:00",
  "day": "2026-04-10",
  "profile": "G1-412837_aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m",
  "variant": "G1-412837",
  "threshold_pct": 12.4,
  "profit_lock_closed_symbols": [],
  "profit_lock_order_type": "market_order",
  "rebalance_order_type": "market",
  "intent_count": 1,
  "orders_submitted": 1,
  "execute_orders": true,
  "g1_diag": {
    "aligned_day": "2026-04-10",
    "weight_c": 0.94,
    "c_symbol": "SOXL",
    "ov_symbol": "SOXL",
    "final_target": {"SOXL": 1.0}
  }
}
```

---

24) Failure and Recovery Runbook

24.1 Failure classes

1. Env/auth failure

- Symptoms: missing key/secret, auth exceptions
- Action: validate `.env.dev`, key names, mode (paper/live), feed entitlement

2. Market/session failure

- Symptoms: market closed loops, calendar errors
- Action: verify timezone and broker clock/calendar responses

3. Data freshness failure

- Symptoms: stale-data skip warnings
- Action: inspect data API latency/feed health; temporarily widen stale threshold only with approval

4. Order lifecycle failure

- Symptoms: rejected orders, insufficient qty, bracket constraints
- Action: inspect broker rejection reason; reconcile current positions; retry only after state consistency check

5. State DB issues

- Symptoms: sqlite lock/corruption/read errors
- Action:
- stop runtime
- back up DB
- validate sqlite integrity
- recover from clean backup if required

24.2 Safe restart protocol

1. Stop process.
2. Snapshot DB file.
3. Inspect latest events:

- confirm last `g1_switch_executed_day`
- confirm whether current day cycle already completed

4. Restart in `--run-once` paper mode first.
5. Validate no duplicate cycle generated.
6. Resume normal mode.

24.3 Duplicate execution prevention check

Before manual re-run same day:

- inspect `g1_switch_executed_day`
- if equals today, do not force second cycle unless explicit replay/test environment

---

25) Paper/Live Operations Checklist

25.1 Pre-open checklist

- [ ] API credentials valid
- [ ] Correct `--mode` selected
- [ ] Correct `--data-feed` selected (`sip` if entitled)
- [ ] State DB path correct and writable
- [ ] System clock and timezone correct
- [ ] No stale orphan process running

25.2 Intraday checklist

- [ ] Intraday slot events are logging every expected interval
- [ ] No repetitive stale-data skips
- [ ] No repeated order rejects
- [ ] Position state matches broker positions

25.3 Eval-time checklist

- [ ] Main cycle fires once
- [ ] `g1_switch_cycle_complete` emitted
- [ ] `orders_submitted` consistent with intent_count and policy

25.4 Post-close checklist

- [ ] End-of-day ticker and weights captured
- [ ] Event count and DB health verified
- [ ] Daily report/export generated

25.5 Kill-switch checklist

- [ ] Stop runtime process
- [ ] Cancel open orders if policy requires
- [ ] Preserve DB snapshot for audit
- [ ] Log reason and timestamp in ops log

---

26) Risk and Compliance Notes

This section is operational guidance, not legal advice.

1. Leveraged ETF concentration risk

- SOXL/SOXS type instruments can move sharply; drawdown can accelerate.

2. PDT / account restrictions

- U.S. rules may constrain frequent same-day round-trips in margin accounts under thresholds.
- Ensure account type and trade frequency policy are compliant.

3. Execution-risk realism

- Runtime-like backtests do not guarantee live fill equivalence.
- Spread, queue position, partial fills, and latency create drift.

4. Order-type-specific risk

- Market order: fill certainty, uncertain price.
- Stop/trailing: may not execute exactly at theoretical trigger.
- Bracket: attach constraints can reject if calculated prices invalid or unsupported.

5. Governance requirement

- Any parameter change should include:
- change proposal
- A/B window impact
- risk sign-off
- rollback plan

---

27) Chart Appendix

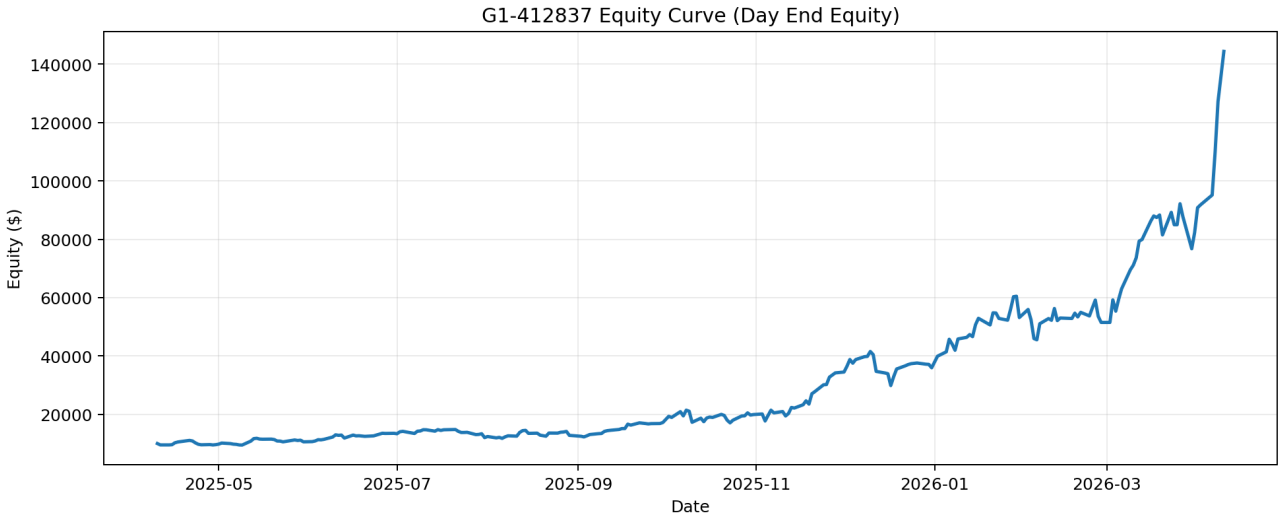
Charts generated from:

- `protective_stop_variant_v2/reports/G1-412837_daybyday_10k_2025-04-10_to_2026-04-10_with_sell_buy_prices.csv`

Generation script:

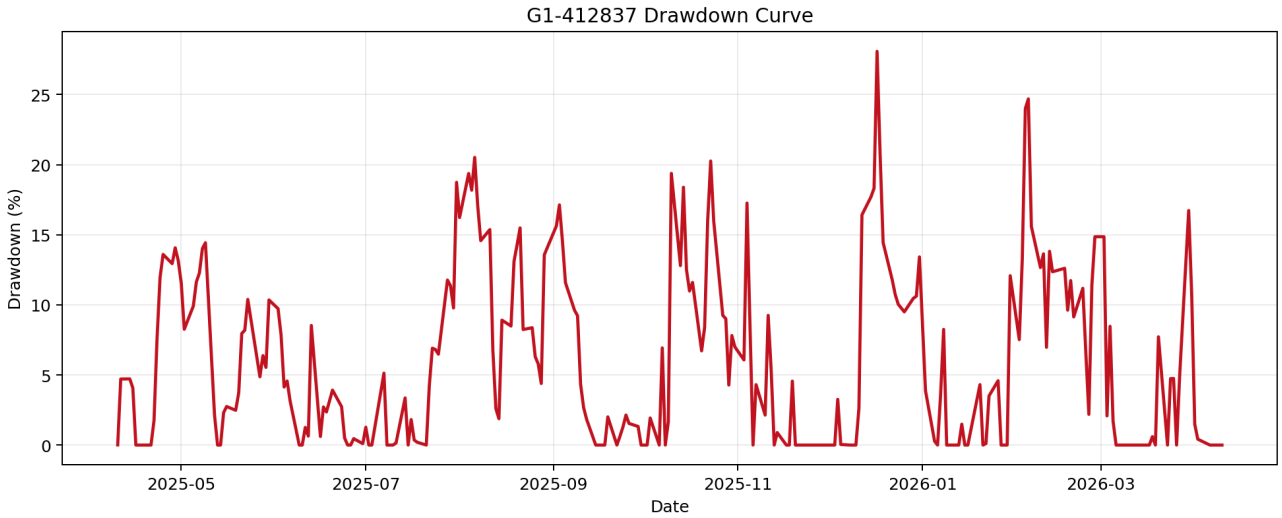
- `g1_412837_runtime_v1/docs/generate_dossier_charts.py`

27.1 Equity curve



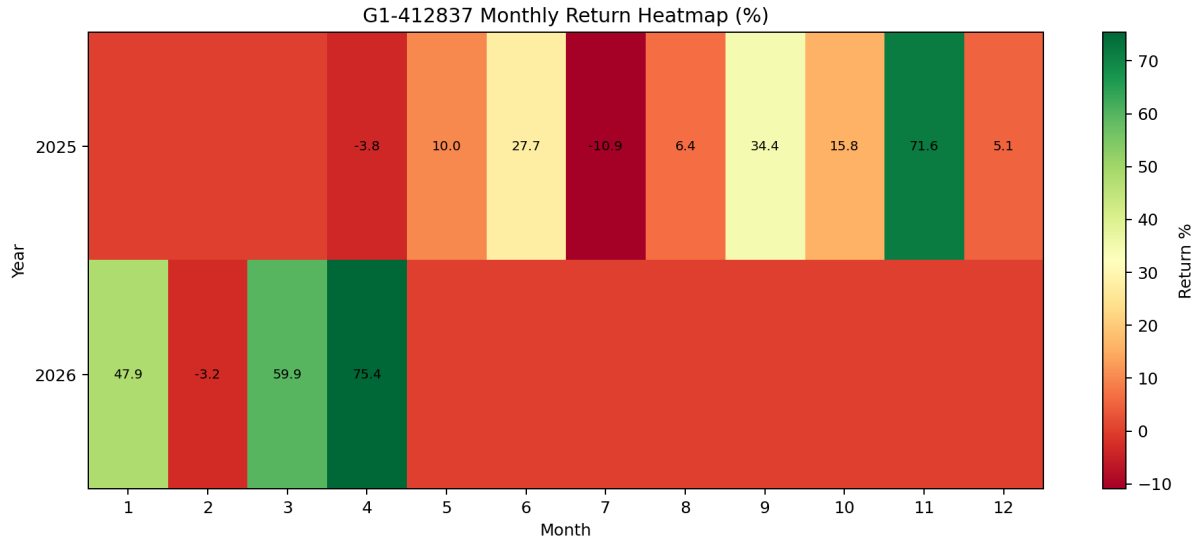
G1 equity curve

27.2 Drawdown curve



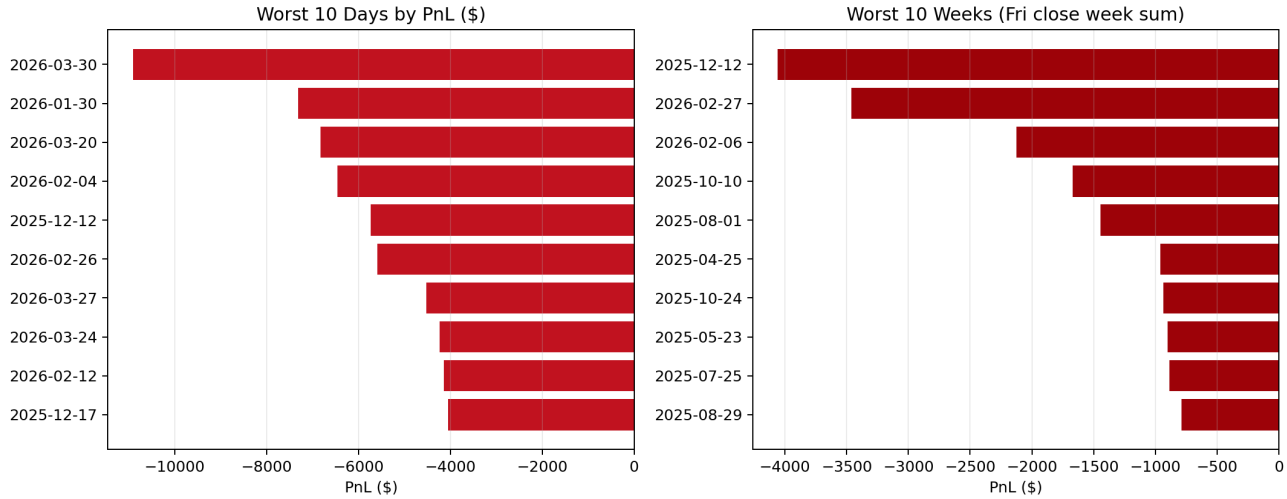
G1 drawdown curve

27.3 Monthly return heatmap



G1 monthly heatmap

27.4 Worst day/week breakdown



G1 worst day week

---

28) Versioned Change Log

```
| Version | Date | Author | Changes |
|---|---|---|---|
| 1.0 | 2026-04-12 | Codex | Initial master dossier (architecture, strategy, parameters, results) |
| 1.1 | 2026-04-13 | Codex | Added full deep-dive for `aggr_adapt_t10_tr2_rv14_b85_m8_M30_intraday_pl_5m` |
| 1.2 | 2026-04-13 | Codex | Added run manifest, provenance/reconciliation, full schema appendix, failure/recove
```

---

29) Pre-Live Go/No-Go Sign-off Template

```
| Item | Owner | Status | Evidence |
|---|---|---|---|
| Manifest complete with SHA + command | Quant Eng |  |  |
| Paper validation completed | Trader/Ops |  |  |
| Risk thresholds approved | Risk |  |  |
| Live credentials and feed verified | Ops |  |  |
| Kill-switch tested | Ops |  |  |
| Rollback plan documented | Quant Eng |  |  |
| Final approval | Strategy Lead |  |  |
```

---

30) Final Notes

This dossier is deliberately detailed and auditable so it can be used as the canonical review packet for:

- strategy audits
- architecture discussions
- paper-to-live readiness gates
- future controlled improvements

For any future revision, preserve:

- document version
- command manifest
- data artifact hash list
- git commit SHA snapshot